Technical source review

Asymptotic

Inverse-coefficient contracts, model limits, validation, and stationary-point branch proofs

A focused delta beyond the existing review register

Prepared for Vladimir Reshetnikov / 10 September 2026

Repository	`VladimirReshetnikov/Asymptotic`
Reviewed commit	`8e859961d7d37f008b826f3a8cad406460271614`
Source baseline	Commit-pinned canonical kernel modules and related development ledgers
Exclusion baseline	The index of 36 reports in `external-reports/code-review`, the consolidated wave-1/2 register, and wave-3/4 intake crosswalks
Evidence	Source inspection; one native package smoke test; 35 independent Python tests; supplied but unexecuted Wolfram integration probes

Executive assessment

This review isolates **three API/validation findings and one bounded mathematical coverage opportunity**. It does not assert a newly reproduced, generally wrong asymptotic expansion. The most concrete defects concern information exposed *about* an inverse: a coefficient's advertised reconstruction omits the source orientation, and a public model field named `Limit` is sometimes an internal normalization offset rather than a limit. A separate, small validation gap admits a purported core inverse that still contains the source variable being eliminated. Finally, both global monotonicity helpers have an avoidable sufficient-condition limitation: strict monotonicity of a polynomial does not require a derivative that is nonzero everywhere.

The fixes are deliberately narrow. Preserve existing normalized coefficients and internal offsets; add explicit observable reconstruction and target-limit metadata. Extend the perturbative argument checks within a documented expression scope. For branch inference, a nonzero polynomial derivative of one weak sign gives a useful, rigorously justified certificate even when it vanishes at finitely many points. The accompanying implementation demonstrates an exact certificate producer and a separate rational-arithmetic checker.

A current native Wolfram kernel successfully loaded the pinned standalone package and expanded $\sin x$ as expected. Subsequent execution-service calls failed, so **none of the new package-level probes is reported as executed**. The 35 successful Python tests establish the independent algebraic models and companion prototype, not current Wolfram or Mathics package acceptance. This distinction is maintained throughout the article and the evidence files.

Contents

1 Scope, evidence, and the non-duplication boundary

1.1 What was inspected

The inspection followed the data path from public request parsing through local-coordinate normalization, sparse power–log jets, inverse construction, observable assembly, and evidence APIs. It also examined selected Gamma/Barnes and elementary exponential carriers, special-function adapters, reciprocal-logarithmic operations, conservative composite envelopes, and Mathics-specific replacement helpers. The source locators in Appendix A identify the main inspected boundaries; this is not a claim that every file in the repository received an exhaustive line-by-line audit.

The authoritative implementation is the canonical modular source, with the root standalone distribution used for the successful native smoke. In particular, the core file owns the ordinary inverse model, final observable assembly, the coefficient accessor, and the perturbative generator. The native and Mathics branch modules own the two global monotonicity helpers discussed below. These source roles can be checked directly in the pinned files.[1, 2, 3, 4]

1.2 What the exclusion process does and does not establish

The repository already contains a substantial review history. Its maintained register maps reports 1–18; the wave-3 and wave-4 intake documents add their respective crosswalks and prose-proposal groups. Those documents, rather than a keyword-only search, were the primary exclusion mechanism. The targeted original ledgers from reports 4 and 6 were also consulted where the proposed deltas approach their API discussions.[6, 7, 8, 9, 10, 11]

The present article does *not* reissue the existing recommendations about generic resource budgets, dense native exports, unknown-remainder calculus, broad result schemas, existing refinement issues, release validation, runner integrity, or general Mathics adapter installation. It also does not rename a known defect by changing its example. The small amount of prior-work discussion below only identifies the boundary separating a new claim from an old one.

Deduplication here is based on the complete maintained crosswalks and their proposal summaries, plus targeted original ledgers. It is not a word-for-word comparison of every earlier article and PDF. Consequently, "delta" means a specific mechanism and witness not found in the inspected register, not an assertion of priority over every sentence ever written about the repository.

1.3 Evidence labels

A *source-established mechanism* is a statement about the inspected definitions and their control flow. A *derived witness* is an exact mathematical evaluation of that mechanism. Neither by itself proves that every public dispatcher reaches that path in a particular interpreter. An *observed package result* requires an actual package execution. A *prototype result* concerns only the supplied companion implementation.

The sole successful native package smoke used the commit-pinned standalone file, whose returned byte count was 679426. Its kernel identified itself as Wolfram Language 15.0.0 for Linux x86 (64-bit), dated 6 May 2026. The returned expression and remainder were

$$\mathrm{Normal}(s) = x - \frac{x^3}{6}, \qquad s[\text{“Remainder”}] = \mathrm{PowerLogRemainder}(x, 5, 0).$$

ID	Present issue	Nearest existing item	Distinguishing condition
F01	Missing source-orientation factor in coefficient reconstruction metadata	C09; D06	No option override or translated endpoint is needed: the identity inverse from below already exposes the discrepancy.
F02	Public model `Limit` is not always the analytic target limit	C10; D02	The witness is `PowerLogModel[1/x,...]` itself, not a residual label or a broad schema proposal.
F03	Core inverse can retain the eliminated source variable	W3-08; D02	A valid-arity `PerturbativeInverse` call is admitted with a structurally invalid core expression.
E01	Global polynomial monotonicity with isolated stationary points	W4-03	This is a whole-interval uniqueness proof, not a deleted-neighborhood radius or eventual-sign search.

Table 1: Novelty boundary. Existing IDs are comparison anchors, not repeated recommendations.

The package reported 38 exported names. These values are preserved as a *manual transcription of the connector result*, not as a freshly machine-verified acceptance receipt.

The later connection failures are not failed mathematical tests. They left the new integration probes unexecuted. The independent local run used Python 3.13.5 and SymPy 1.14.0, with 35 tests, zero failures, zero errors, and zero skips. Its transcript and JSON summary are included separately. Historical test counts quoted in repository documentation were not added to that total and were not relabeled as current evidence.[12]

2 The relevant architecture: model, chart, and observable

The ordinary engine works in a positive local coordinate $u \to 0^+$. At a finite source endpoint, write

$$x = x_0 + \sigma u, \qquad \sigma \in \{-1, 1\};$$

at an infinite endpoint, write

$$x = \frac{\sigma}{u}.$$

The sign is an essential part of the chart. It is not, in general, a coefficient of the normalized inverse unit.

A normalized finite power–log model has the form

$$f(x(u)) = b + a u^p \left(1 + \sum_{i=1}^{m} u^{d_i} P_i(\log u)\right), \qquad d_i > 0, \tag{1}$$

with a nonzero real leading coefficient a and nonzero real leading power p. In the actual source, `rowsToModel` stores the extracted baseline b, the leading data, the gaps, and the logarithmic coefficient polynomials. The inverse constructor subsequently decides the actual target endpoint and assembles the observable with the source sign.[1]

For finite target endpoint y_0, let $v = y - y_0$; for an infinite target endpoint, the constructor uses $v = y$. The positive uniformizer is

$$z = (v/a)^{1/p}, \qquad L = \log(v/a)/p.$$

For a source observable of power r, define

$$\rho = \begin{cases} r, & x_0 \text{ finite}, \\ -r, & x_0 \text{ infinite}. \end{cases}$$

The coefficient machinery computes contributions of the normalized local quantity u^ρ, not automatically of the final signed source observable.

This organization is mathematically natural. It separates positive-coordinate power algebra from orientation and from additive translation. The review therefore does *not* recommend changing the coefficient algebra to carry a sign redundantly through every recurrence. The problem is at the public description boundary: the returned metadata must say which of these quantities a coefficient reconstructs.

The same separation matters for proof records. A certificate of global injectivity, a lower bound on a derivative, and an asymptotic error estimate are different objects. An extension that proves the first must not be substituted wherever an existing consumer requires the second. This distinction becomes decisive in Section 6.

3 F01: inverse coefficient metadata drops source orientation

3.1 Finding and severity

Classification: medium-priority API correctness and reconstruction contract. The ordinary inverse constructor multiplies the assembled local expansion by σ^r. The object overload of `InverseExpansionCoefficient` adjusts the internal power for an infinite source endpoint, but does not attach the corresponding orientation multiplier to its returned coefficient or its `Meaning` field. Its meaning describes a contribution of the form

$$(v/a)^\beta \, C(L),$$

without the factor needed to interpret that contribution as part of the represented observable.[1]

The normalized coefficient itself need not be wrong. A caller deliberately requesting a coefficient of the positive local-coordinate expression may want exactly that value. The defect is that the public object-oriented accessor does not make the missing chart factor explicit, even though the object it was called on already represents the signed source observable.

3.2 Minimal witness: the identity inverse from below

Consider the public probe

```
s = AsymptoticInverse[x, {x, 0}, {y, 2},
  Direction -> "FromBelow"];
c = InverseExpansionCoefficient[s, {}];
{Normal[s], c}
```

This probe is supplied for execution; the following values are the exact source-derived prediction, not a newly observed kernel transcript.

The local coordinate is $u = -x$, so $f = -u$, $a = -1$, $p = 1$, and $\sigma = -1$. There are no correction gaps and the leading multi-index is empty. The coefficient routine returns the leading normalized coefficient $C = 1$ at exponent $\beta = 1$. Its described local contribution is therefore

$$(y/(-1))^1 \cdot 1 = -y.$$

The inverse constructor, correctly, multiplies this by $\sigma = -1$, obtaining

$$\mathrm{Normal}(s) = y.$$

Thus a caller following the coefficient record's advertised reconstruction without separately rediscovering the source chart obtains the negative of the represented inverse. No approximate arithmetic, difficult branch theorem, conflicting option, or finite translation is needed.

The same issue occurs at the negative infinite source endpoint. For $f(x) = x$, $x \to -\infty$, one has $u = -1/x$, $a = -1$, $p = -1$, and $\rho = -1$. The normalized contribution is again $-y$, while the observable is y.

3.3 The discrepancy affects nonleading terms as well

For $f(x) = x + x^2$ near zero from below,

$$f(-u) = -u + u^2 = -u(1 - u).$$

The normalized inverse has positive Catalan coefficients in powers of $-y$:

$$u = -y + y^2 - 2y^3 + 5y^4 + \cdots .$$

The source inverse is

$$x = -u = y - y^2 + 2y^3 - 5y^4 + \cdots .$$

In particular, the local contribution at the first nontrivial multi-index is $+y^2$; the observable contribution is $-y^2$. A leading-term-only documentation example would not fully exercise the contract.

For an even integral observable power, $\sigma^r = 1$, so this witness disappears. For an odd integral power, it persists. That parity distinction is a useful positive control: a proposed repair that indiscriminately negates every coefficient on a negative source branch is itself wrong.

3.4 A precise reconstruction law

Let the coefficient routine return $\beta_{\mathbf{k}}$ and $C_{\mathbf{k}}(L)$. The contribution to the source observable is

$$T_{\mathbf{k}}(y) = \sigma^r (v/a)^{\beta_{\mathbf{k}}} C_{\mathbf{k}}\left(\frac{\log(v/a)}{p}\right). \tag{2}$$

For a finite endpoint and $r = 1$, add x_0 *once* to the sum of these terms. For finite endpoints and other admitted powers, the ordinary constructor's output is the powered displacement, not an extra copy of the endpoint at every coefficient. For infinite source endpoints there is no additive source offset. These are the actual assembly cases in the inspected constructor.[1]

Equation (2) must be applied before interpreting a coefficient as an observable contribution. Equal-weight multi-indices may then be collected. The orientation repair does not change which multi-indices collide, how truncation is requested, or how a remainder is justified.

3.5 Recommended compatible repair

Do not silently redefine the existing `Coefficient` field. Add fields that preserve both interpretations:

```
"LocalCoefficient"      -> C
"ObservableMultiplier"  -> sigma^r
"ObservableCoefficient" -> sigma^r C
"AdditiveOffset"        -> sourceOffset
"ContributionExpression" -> orientedTermInTargetVariable
```

The `Meaning` text should explicitly distinguish a normalized local coefficient from an observable contribution. The additive offset must be documented as a once-per-expansion baseline. The example should show both a right-source control and a left-source witness.

The supplied `OrientedInverseContribution` helper implements this as an additive API, without modifying package definitions. Its scope is deliberately restricted to ordinary, exponent-truncated inverse objects. It does not pretend that Gamma/Barnes, composite, native, logarithmic, or raw reconstructed associations all satisfy the same coefficient contract.

3.6 Acceptance conditions and non-overlap

A focused acceptance selection should cover the identity from both finite sides, both infinite endpoints, one translated finite endpoint, odd and even integral powers, and one nonleading contribution. Summing the new contribution expressions must agree with the same retained local terms after the constructor's existing chart transformation; the additive offset is added exactly once.

This is distinct from C09, where an explicit `Power` option is shadowed by stored data: the witness supplies no override. It is also narrower than D06's discussion of source-point versus displacement observables: the minimal witness uses $x_0 = 0$, so no translation convention can explain away the missing sign.[7, 11]

4 F02: the public model's Limit field can be an offset

4.1 Finding and severity

Classification: medium-priority metadata semantics. The public `PowerLogModel` function exposes the association returned by `rowsToModel`. The latter initializes its `Limit` value to zero and extracts a constant baseline only when the leading row has weight zero. When the leading power is negative, that stored value is not the analytic limit of the source expression.[1]

The inverse constructor later changes its *own* target endpoint to the appropriate signed infinity. That correction does not turn the model's original field into a target-limit field. Consequently, model and inverse metadata use the same word for different roles.

4.2 Minimal and diagnostic witnesses

The public probe

```
m = PowerLogModel[1/x, {x, 0}];
{m["Limit"], m["LeadingPower"], m["LeadingCoefficient"]}
```

has the source-derived prediction $\{0, -1, 1\}$, although the admitted positive local approach has

$$\lim_{x \to 0^+} \frac{1}{x} = +\infty.$$

For $-1/x$, the limit is $-\infty$, while the normalized model baseline is again zero. For $7 + x$, the extracted baseline and actual limit are both seven, so ordinary regular examples conceal the distinction.

A particularly useful control is

$$f(x) = \frac{1}{x} + 7.$$

Here the leading row has weight -1; the constant seven remains in the correction rows rather than being removed as the baseline. Therefore the model's stored `Limit` is zero, the Laurent constant coefficient is seven, and the actual limit is positive infinity. These are *three different quantities*. Renaming the old field to "constant term" would not solve the semantic problem.

4.3 Why the normalized offset is legitimate internal data

The model decomposition (1) still represents the source correctly. When $p < 0$, a finite constant may be treated as a subleading relative correction. That is a valid algebraic normalization. For $1/u + 7$, one may write

$$\frac{1}{u} + 7 = 0 + u^{-1}(1 + 7u).$$

It would be destructive to replace the baseline in this expression by infinity merely because the old key is named `Limit`. Internal consumers use the baseline algebraically. A repair must separate meanings rather than corrupt the model to match a label.

For the admitted model class, the target limit follows directly from the leading data:

$$\text{TargetLimit} = \begin{cases} b, & p > 0, \\ +\infty, & p < 0 \text{ and } a > 0, \\ -\infty, & p < 0 \text{ and } a < 0. \end{cases} \tag{3}$$

If an implementation cannot establish the necessary sign under its retained assumptions, it should return an explicit unresolved value rather than guess. The public ordinary model currently admits numerical real exponents; a future symbolic extension must maintain that distinction.

4.4 Recommended repair and migration

Introduce `ModelOffset` for the baseline extracted by normalization and `TargetLimit` for the analytic endpoint. Preserve the existing field as a documented legacy alias during a compatibility period, or

add a versioned explicit accessor that returns both meanings. The supplied `DescribeModelLimit` companion demonstrates the latter approach.

For internal cleanup, update baseline consumers to use `ModelOffset` before retiring the ambiguous name. The exact-termination check is one example: it constructs a finite target expression from the model baseline and leading power. Such a consumer must not accidentally read a signed-infinite endpoint.[5]

For public documentation, show at least one pole and one regular finite limit. State explicitly that `ModelOffset` is neither automatically the Laurent constant coefficient nor the target endpoint. This resolves a concrete ambiguity without requiring a wholesale new object schema.

4.5 Acceptance conditions and non-overlap

Test x, $7 + x$, $1/x$, $-1/x$, and $7 + 1/x$ on the appropriate real approaches. The old normalized reconstruction must remain algebraically unchanged. The new target-limit field must agree with the leading-data theorem, and the inverse object's target endpoint must agree with it whenever both are available.

C10 concerns wording in the residual normalization; F02 concerns the actual value exposed by a different public API before any residual is computed. D02's broader shared-contract discussion is not re-proposed here. The implementation delta is the explicit separation of these two particular fields.[7]

5 F03: a core inverse may retain the eliminated source variable

5.1 The intended formal contract

The four-argument `PerturbativeInverse` accepts a core inverse $\phi(y)$, a source perturbation $h(x)$, source and target variables, and a nonnegative order. Its native implementation computes

$$\phi(y) + \sum_{k=1}^{n} \frac{(-1)^k}{k!} \frac{d^{k-1}}{dy^{k-1}} \left[\phi'(y) h(\phi(y))^k\right]. \tag{4}$$

The Mathics replacement evaluates the same finite family using `Table` and `Total` instead of the native symbolic `Sum` path. Both definitions check that source and target symbols differ and that h does not contain the target variable. Neither checks that the purported core inverse is free of the source variable.[1, 4]

5.2 The admitted malformed call

Consider

```
PerturbativeInverse[x + y, x^2, {x, y}, 1]
```

Both existing argument tests succeed. Substitution gives $h(\phi) = (x + y)^2$, and differentiation with respect to y gives the formal result

$$x + y - (x + y)^2.$$

The supposedly eliminated source symbol survives. This is not a counterexample to the Lagrange–Buermann formula on valid data; it is an argument-admission gap that lets a structurally invalid core pass as an ordinary result.

Classification: low-priority validation and diagnostic quality. No claim is made that a valid core inverse is expanded incorrectly by this example. The repair prevents a confusing successful-looking output rather than repairing the mathematical recurrence.

5.3 Minimal repair and the scope caveat

For the same structural expression policy already used for h, the minimal check is

```
If[x === y || ! FreeQ[h, y] || ! FreeQ[phi, x],
  Failure["InvalidVariables", <|"MessageTemplate" ->
    "Use distinct symbols; h must not contain y and phi must not contain x."|>],
  (* existing computation *)
]
```

It must be applied to both the native definition and the late Mathics replacement. Patching only the native definition leaves the second runtime's admission unchanged. The included Python patch generator checks exact old fragments in both files and emits a unified diff without modifying the checkout.

There is a subtle boundary to document. `FreeQ` is a structural occurrence check, not a complete lexical free-variable analysis for every Wolfram binder. A core containing a *bound* occurrence of a same-named symbol inside an unevaluated scoping construct is not mathematically the same as $\phi = x + y$. The small companion therefore adopts an explicit-expression structural policy matching the existing perturbation check. If binder-containing cores are part of the supported contract, the final production repair should use scope-aware free-variable analysis or alpha-renaming rather than rejecting all bound occurrences blindly.

This validation does not prove that ϕ really inverts an arbitrary undisclosed core function; no such core function is supplied to this API. It also does not manufacture an asymptotic remainder theorem for the finite perturbation formula.

5.4 Positive controls

For $\phi(y) = y$ and $h(x) = ax^2$, formula (4) gives

$$y - ay^2 + 2a^2y^3 - 5a^3y^4$$

through third perturbative order. For $\phi(y) = \sqrt{y}$ and $h(x) = ax^2$, it gives

$$\sqrt{y}\left(1 - \frac{a}{2} + \frac{3a^2}{8} - \frac{5a^3}{16}\right),$$

which agrees with the corresponding finite expansion of $\sqrt{y/(1+a)}$. These exact controls, the malformed witness, the zeroth-order validation case, and the existing target-variable restriction are covered by the independent Python tests.

The supplied Wolfram helper leaves all package definitions untouched. Its native and Mathics integration remain unexecuted. The patch generator itself is intentionally a preparatory tool: after accepting its diff, regenerate the standalone distribution using the repository's existing process rather than independently editing generated source.

6 E01: global monotonicity through isolated stationary points

6.1 Current sufficient condition and its limit

The native `inverseBranchGlobalMonotonicity` first checks continuity and convexity, then tries to prove a derivative strictly positive everywhere or strictly negative everywhere. Its additional special logarithmic lemma does not change the polynomial case considered here. The Mathics polynomial/convex-domain wrapper likewise accepts a strict derivative sign and otherwise returns `None`.[2, 3]

That is a sound sufficient condition. It is not necessary for strict monotonicity. This section proposes a bounded coverage extension, not a correction to an unsound existing proof. A failed helper certificate also does not establish that the full public inverse dispatcher fails: another branch-completeness route may succeed. The supplied probe targets the helper explicitly, and public integration must be tested separately.

6.2 A compact exact witness

Let

$$F(x) = x - \frac{2}{3}x^3 + \frac{1}{5}x^5. \tag{5}$$

Then

$$F'(x) = (x^2 - 1)^2 \geq 0,$$

with zeros at $x = \pm 1$. Thus the current strict-derivative condition is false on $\mathbb{R}$. Nevertheless, for any $a < b$,

$$F(b) - F(a) = \int_a^b (t^2 - 1)^2 \, dt > 0.$$

The integrand is nonnegative and can vanish only at two points, not on a nontrivial interval. Therefore F is strictly increasing. Its odd degree and positive leading coefficient give limits $-\infty$ and $+\infty$, so it is a bijection of $\mathbb{R}$.

The example avoids relying on native simplification of `InverseFunction[Function[t,t^3]]` to a familiar root expression. More importantly, it isolates the missing theorem: a derivative can have zeros without creating a second inverse branch.

Proposition 6.1 (Polynomial weak-sign certificate)**.** *Let $I \subseteq \mathbb{R}$ be a nondegenerate interval and $F \in \mathbb{R}[x]$. If F' is not the zero polynomial and $F'(x) \geq 0$ for all $x \in I$, then F is strictly increasing on I. With the reversed derivative inequality it is strictly decreasing.*

Proof. A nonzero polynomial has finitely many roots. For any $a < b$ in I, the interval (a, b) therefore contains a point where $F' > 0$. Continuity makes F' positive on a smaller interval of positive length. Its integral over $[a, b]$ is strictly positive, while the rest of the integral is nonnegative. The decreasing case follows by applying the same argument to $-F$. □

The interval assumption is essential. A derivative of one sign on two disjoint pieces does not by itself compare function values across the gap. The proposed first implementation therefore handles the entire real line, where convexity is built into the supported domain, rather than pretending to solve arbitrary Boolean domain geometry.

6.3 An exact rational-polynomial decision procedure

For a rational polynomial F, set $Q = F'$. If $Q = 0$, no strict-monotonicity certificate is issued. Otherwise write a square-free decomposition

$$Q = c \prod_{j=1}^{s} q_j^{m_j} = c\, A^2 B,$$

where

$$A = \prod_j q_j^{\lfloor m_j/2 \rfloor}, \qquad B = \prod_{m_j \text{ odd}} q_j.$$

The factors are rational polynomials, and the product identity can be checked exactly. The whole-line sign test reduces to determining whether B has a real root. If it has none, B has constant sign; consequently Q has one weak sign and is nonzero except at finitely many roots of A. Its sign away from those roots is obtained from the leading coefficient of Q. The proposition then certifies strict monotonicity.

Conversely, if a square-free odd-multiplicity part has a real root, Q changes sign there, so this whole-line weak-sign route cannot certify monotonicity. The prototype returns `NoCertificate` instead of silently weakening its conclusion. The distinction between a local interval and the entire line is maintained.

For (5),

$$Q = (x^2 - 1)^2, \qquad c = 1, \quad A = x^2 - 1, \quad B = 1.$$

No nontrivial root isolation is even needed for the odd part. The certificate records two stationary real roots while explicitly asserting no positive global derivative lower bound.

6.4 Sturm sequences and a separately checkable certificate

The optional producer uses SymPy's square-free polynomial factorization. The official polynomial interface documents both square-free factor lists and Sturm sequences. The companion checker does not trust the producer's factorization, root count, or claimed direction; it recomputes the relevant identities using rational coefficients and exact polynomial division.[13]

For a nonzero polynomial B, form

$$S_0 = B, \qquad S_1 = B', \qquad S_{i+1} = -\operatorname{rem}(S_{i-1}, S_i)$$

until the next remainder is zero. A constant B uses a one-element chain. The root count is obtained from the difference of sign variations at $-\infty$ and $+\infty$. At these endpoints, the sign of a polynomial is determined exactly by its leading coefficient and degree; no numerical approximation to a root is needed.

The emitted JSON contains the input polynomial, its derivative, the factor coefficient, polynomial factors and multiplicities, the odd part, its Sturm chain, the claimed monotonicity direction, and a stationary-root count. Verification checks the derivative identity, factor product, odd-part product,

entire remainder recurrence, root count, and conclusion. It additionally recomputes the derivative's distinct real-root count for the stationary-point metadata.

```
python code/polynomial_certificate.py produce \
  '["0","1","0","-2/3","0","1/5"]'

python code/polynomial_certificate.py verify \
  evidence/quintic-monotonicity-certificate.json
```

The input convention is ascending powers. Thus the first command represents exactly $x - 2x^3/3 + x^5/5$. No expression string is evaluated as Python or Wolfram code.

6.5 Strict injectivity is not a slope bound

The new certificate must have a different type from `StrictDerivativeOnRealInterval`. In particular, it must not carry an invented positive `DerivativeLowerAbsoluteBound`. The function in (5) has zero infimum of $|F'|$ on any interval containing either stationary point.

At $x = 1$, the exact local identity is

$$F(1 + h) - \frac{8}{15} = \frac{4}{3}h^3 + h^4 + \frac{1}{5}h^5. \tag{6}$$

Writing $\delta = y - 8/15$, the inverse has leading displacement

$$h \sim \sqrt[3]{\frac{3}{4}\delta}.$$

It is uniquely defined but is not locally Lipschitz at the stationary target. A small forward residual translates into a cube-root scale there, not a residual divided by a positive uniform slope. This is why a global branch-uniqueness result cannot simply be inserted into numerical inverse-error code as a derivative estimate.

A suitable proof record would identify its type as `WeakDerivativeWithFiniteZeroSet`, retain the domain, derivative and sign, and explicitly distinguish global injectivity from local inverse order. Existing local branch validation and asymptotic construction should continue to establish the source approach, target side, and local scaling.

6.6 Integration route and scope

A minimal integration sequence is to keep the current cheap strict-slope checks first, then try the bounded rational-polynomial route only when they are inconclusive. On native Wolfram, an explicitly strict `FunctionMonotonicity` query is another possible characterization control; its documentation distinguishes strict and non-strict requests. Such a query should specify the strictness option rather than relying on a default. No current output of that query is claimed here.[14]

For Mathics, the mathematical algorithm can be ported using exact polynomial arithmetic; the supplied Python producer/checker is not already wired into the package. Installing a subprocess dependency or trusting an external certificate without verification would be a separate engineering decision. This article does not claim a Mathics implementation has been completed.

The first scope should remain rational coefficients on $\mathbb{R}$. Parameter-dependent leading cancellations, algebraic coefficient fields, and arbitrary disconnected source domains introduce additional proof obligations. Those are not prerequisites for accepting the quintic witness, and they should not inflate a small branch-coverage change into a broad theorem-prover redesign.

6.7 Why this is not the existing eventual-neighborhood proposal

W4-03 concerns finding a sufficiently small deleted neighborhood and carrying a justified radius through later local proofs. E01 concerns strict ordering of every pair of points in one whole interval despite isolated derivative zeros. It neither chooses a smaller radius nor improves the existing seven-radius search. It supplies a different theorem with a different consumer contract.[9]

7 Performance implications, without unsupported speed claims

7.1 F01 and F02 need no expensive reconstruction

The orientation factor in F01 is already determined by the stored source endpoint, direction and observable power. Attaching it to coefficient metadata does not require recomputing inverse coefficients or replaying the source function. Likewise, formula (3) uses the model's existing leading data; it does not require a new general symbolic `Limit` call for each public model.

For numeric real parameters already admitted by the ordinary model, both operations are small compared with inverse construction. This is an algorithmic work observation, not a measured end-to-end speedup. In particular, the companion implementation sometimes invokes simplification for its presentation, and that cost should not be confused with the minimum production metadata cost.

7.2 F03 rejects before differentiation

The malformed-core guard can run before substitution and before any of the higher derivatives in (4). On an admitted explicit expression tree, a structural occurrence test is linear in the expression's visited structure. The correction is worthwhile chiefly for clarity; no claim of a significant performance regression or improvement on valid inputs is made.

A broader scope-aware validator would have a different cost and contract. It should be introduced because binder-containing cores are supported and need it, not because the present malformed witness somehow proves every syntactic occurrence check sufficient.

7.3 E01 moves work to a bounded algebraic question

The certificate route replaces an impossible strict-derivative proof for the witness with a decidable algebraic sign question. This is an availability improvement. It may also avoid repeated general symbolic proof attempts for the same polynomial, but that must be measured on the actual package rather than inferred from the Python prototype.

The implementation bounds polynomial degree at 64, coefficient numerator/denominator bit lengths at 4096, and serialized certificate size at 250000 bytes. It checks projected factor-product degree before

raising factors to their multiplicities. Those are explicit prototype admission limits, not proposed replacements for the repository's existing resource-budget design.

With naive dense polynomial arithmetic, one multiplication uses quadratically many rational coefficient operations in the degree. A Euclidean remainder sequence performs a bounded number of divisions with decreasing degrees. Coefficient bit growth can dominate these operation counts, so a claim of merely "polynomial time in degree" would omit an important parameter. Production profiling should record degree, coefficient size, factorization time, remainder-chain length, and verification time separately for this feature.

The checker uses dense coefficient arrays only inside its explicitly degree-bounded prototype. It is not proposed as a new representation for the repository's sparse asymptotic jets. Nor is it an operating-system sandbox: input limits and rational identities do not replace a process-level deadline for a certificate producer doing expensive factorization.

8 Implementation sequence and focused acceptance

8.1 Small compatible changes first

First establish a concrete coefficient-reconstruction contract and add the F01 fields or an equivalent documented accessor. Keep local coefficients unchanged. Next split the F02 model baseline and target endpoint, migrating internal baseline consumers before deprecating the ambiguous field. These changes should be reviewable without altering any coefficient recurrence, branch choice, or asymptotic remainder.

The F03 guard is independent of those metadata changes. Apply it to both definitions under a stated expression policy, regenerate the standalone, and use valid-core controls to detect accidental behavior changes. Do not count an expected refusal as evidence that the perturbative formula itself was improved.

Then implement E01 as a separate proof type with a narrow rational-polynomial admission predicate. Its acceptance must establish both that the witness is now certifiable and that no consumer mistakes weak-sign injectivity for a positive slope bound. This last condition is at least as important as making the example succeed.

8.2 Expected regression matrix

Item	Positive and negative controls	Acceptance property
F01	Identity from both finite sides; both infinities; a translated endpoint; odd-/even powers; a correction term	Observable contributions reconstruct the represented finite expression after adding the offset once. Local coefficients and remainder metadata are unchanged.
F02	Regular limit, positive/negative pole, pole plus a constant, unresolved sign	Model reconstruction is unchanged. Target endpoint and normalization baseline are separate; no consumer performs arithmetic with an infinite endpoint where it needs a baseline.

Item	Positive and negative controls	Acceptance property
F03	Identity core, square-root core, source-containing core, target-containing perturbation, order zero	Valid controls preserve their formulas. Invalid core data is diagnosed before differentiation. Native and Mathics definitions have the same admitted expression policy.
E01	Strict-slope polynomial, cube, quintic with two stationary points, negative quintic, nonmonotone quadratic, constant	Only justified strict-monotonicity records are accepted. Stationary zeros are not represented as a positive slope lower bound. Unknown or out-of-scope cases retain existing fallback behavior.

8.3 The artifacts and their actual status

`exact_models.py` contains independent expressions for F01–F03. `polynomial_certificate.py` contains the optional producer and standard-library checker for E01. `test_review.py` executes the 35 independent tests and writes the local evidence record. The tests include tampering with direction, factor coefficient, derivative, Sturm recurrence, stationary-root count, multiplicity, and domain.

`ReviewDeltas.wl` contains additive companion helpers for F01–F03, without changing the package. `native_probes.wl` loads a caller-specified package path in a fresh kernel and records bounded observations. `prepare_validation_patch.py` emits the two-file F03 candidate diff after checking exact baseline fragments. The Wolfram helpers and probes have not been executed in the current review environment.

The native probe runner records the expected commit but does not independently verify that the supplied file came from it. The operator must use the pinned checkout. This explicit limitation prevents a user-supplied file path from being mistaken for verified source provenance.

8.4 What remains unestablished

No current public integration failure has been reproduced for the new witnesses. No full native suite, Mathics suite, benchmark matrix, or production patch acceptance has been executed for this article. The source analysis establishes the narrow mechanisms, while the exact derivations and Python tests establish their mathematical models. The remaining public-dispatch and interpreter-specific questions are represented by executable probes, not by invented output.

The review also does not certify the absence of other bugs in inspected modules. Several candidate paths were deliberately not promoted to findings when their guards were present, their conservative result was mathematically valid, or their underlying issue was already in the review register. The absence of a new allegation in such an area is not a completeness theorem about it.

9 Conclusion

The useful next work identified here is smaller and more concrete than another broad architecture rewrite. The ordinary inverse's internal separation of positive local coordinates and final source observables should become explicit in its public coefficient data. A public model should expose both its

normalization baseline and its actual target limit without overloading the word "limit." A perturbative generator should reject an explicit core that still contains the source symbol it is meant to eliminate. And polynomial branch inference can admit isolated stationary points without weakening the truth of its uniqueness claim.

These changes have different evidentiary weights. F01 and F02 are source-supported metadata inconsistencies with exact minimal witnesses. F03 is a malformed-input validation gap, not an error on valid mathematical data. E01 is a sound bounded extension, with a tested independent prototype but no installed package implementation. Keeping those distinctions visible makes the report actionable without inflating a small number of defensible deltas into a misleading catalog of new numerical failures.

A Source locators and inspection map

The following paths refer to the pinned commit, not to a moving branch. Function names are the primary locators; line ranges in the companion ledger identify inspected source windows rather than asserting a complete-file audit.

Path / boundary	Inspected role
`src/Kernel/AsymptoticAnalysis.wl`	Exact-number ordering, coefficient normalization, sparse jets, forward assembly, `rowsToModel`, ordinary inverse construction, object formatting, coefficient accessor, perturbative generator.
`SeriesOperations.wl`	Representation conversion, arithmetic, observables, composition, differentiation, refinement replay.
`SeriesArithmetic.wl` and `SeriesEnvelopeArithmetic.wl`	Held arithmetic admission and conservative mixed-scale error transport.
`InverseFunctionSyntax.wl` and `InverseFunctionBranches.wl`	Callable/condition parsing, global monotonicity, deleted-neighborhood branch validation.
`InverseCertificates.wl` and `NumericalInverseChecks.wl`	Rational interval evidence and numerical inverse comparison boundaries.
`GammaInverseChecks.wl`, `GammaInverseOperations.wl`	Original logarithmic equation checks, formal residuals, observable powers and inherited precision.
`BarnesForward.wl`, `ExponentialForward.wl`	Exact logarithmic normalization and retained asymptotic carriers.
`ParameterizedSpecialFunctions.wl`, `DirichletSpecialFunctions.wl`, `SpecialFunctionAdapters.wl`	Selected defining-series, large-parameter, and threshold-adapter contracts.
`ReciprocalLogOperations.wl`, `ExactTermination.wl`	Analytic reciprocal-log calculus and verification against original inverse sources.
`MathicsInverseBranches.wl`, `MathicsCalculus.wl`	The E01 weak-sign coverage boundary and the F03 late replacement definition.
Other inspected Mathics helpers	Assumptions, algebra, simplification, calls, compatibility, time budgets, Taylor providers, ProductLog construction, Stirling providers, and certificate assignment compatibility.

B Reproduction commands

Run from the archive's top-level directory. The independent test suite needs Python and SymPy; the certificate verifier itself uses only the standard library.

```
python code/test_review.py
python code/polynomial_certificate.py verify \
  evidence/quintic-monotonicity-certificate.json

# Native observations, in a fresh Wolfram kernel:
wolframscript -file code/native_probes.wl \
  /absolute/path/to/pinned/AsymptoticAnalysis.wl

# Emit, but do not apply, the F03 candidate patch:
python code/prepare_validation_patch.py \
  /absolute/path/to/pinned/Asymptotic > validation.patch

# Rebuild this article:
cd article
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

References

[1] VladimirReshetnikov/Asymptotic, *Canonical kernel core*, reviewed commit. src/Kernel/AsymptoticAnalysis.wl.

[2] VladimirReshetnikov/Asymptotic, *Real inverse-function branch inference*, reviewed commit. src/Kernel/InverseFunctionBranches.wl.

[3] VladimirReshetnikov/Asymptotic, *Mathics inverse-branch helpers*, reviewed commit. src/Kernel/MathicsInverseBranches.wl.

[4] VladimirReshetnikov/Asymptotic, *Mathics finite calculus replacements*, reviewed commit. src/Kernel/MathicsCalculus.wl.

[5] VladimirReshetnikov/Asymptotic, *Exact inverse termination verification*, reviewed commit. src/Kernel/ExactTermination.wl.

[6] VladimirReshetnikov/Asymptotic, *External review index*, reviewed commit. external-reports/code-review/README.md.

[7] VladimirReshetnikov/Asymptotic, *Consolidated review status and crosswalk*, reviewed commit. docs/development/CODE_REVIEW_STATUS.md.

[8] VladimirReshetnikov/Asymptotic, *Wave-3 intake and finding crosswalk*, reviewed commit. docs/development/WAVE_3_INTAKE.md.

[9] VladimirReshetnikov/Asymptotic, *Wave-4 intake, detailed obligations, and proposal groups*, reviewed commit. docs/development/WAVE_4_INTAKE.md.

[10] VladimirReshetnikov/Asymptotic, *Report 4 original finding ledger*, reviewed commit. wave-1/code-review-4/evidence/findings.json.

[11] VladimirReshetnikov/Asymptotic, *Report 6 original finding ledger*, reviewed commit. wave-1/code-review-6/evidence/findings.csv.

[12] VladimirReshetnikov/Asymptotic, *Review and validation record*, reviewed commit. validation/README.md.

[13] SymPy developers, *Polynomials Manipulation Module Reference*, documentation consulted 10 September 2026; square-free factorization and Sturm-sequence interfaces. https://docs.sympy.org/latest/modules/polys/reference.html.

[14] Wolfram Research, *FunctionMonotonicity*, documentation consulted 10 September 2026; explicit strictness option. https://reference.wolfram.com/language/ref/FunctionMonotonicity.html.